pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

The Daily Market Snapshot

WEEK 3 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: one page, any date

The Desk Lead's Note

Before every morning meeting I want one page. Where the curve and our two spread series closed on the last market date, how they moved over a day, a week and a month, and where each sits in its own year. The curve's shape. What the day's curve move does to the September book, as a what-if. A web page I can open and a workbook I can check in Excel. Then make it one command for any date, set it to run every morning, and keep the script in our record, never its pages. A page for 30 September shows 30 September, even when the file runs on.

The spreads, the book and its issuers are synthetic. Every sentence on the page describes, and none forecasts. Every number comes from the course's files, so there is no key and no live cell.

```
Work folder pfi_course3_14_k4w7q2xr: 8 data files, Course 2's bondmath.py, marketdata.py as notebook 13 left it, 13 commits, tagged week1 and week2
.................................................   [100%]
51 passed in 0.50s
```

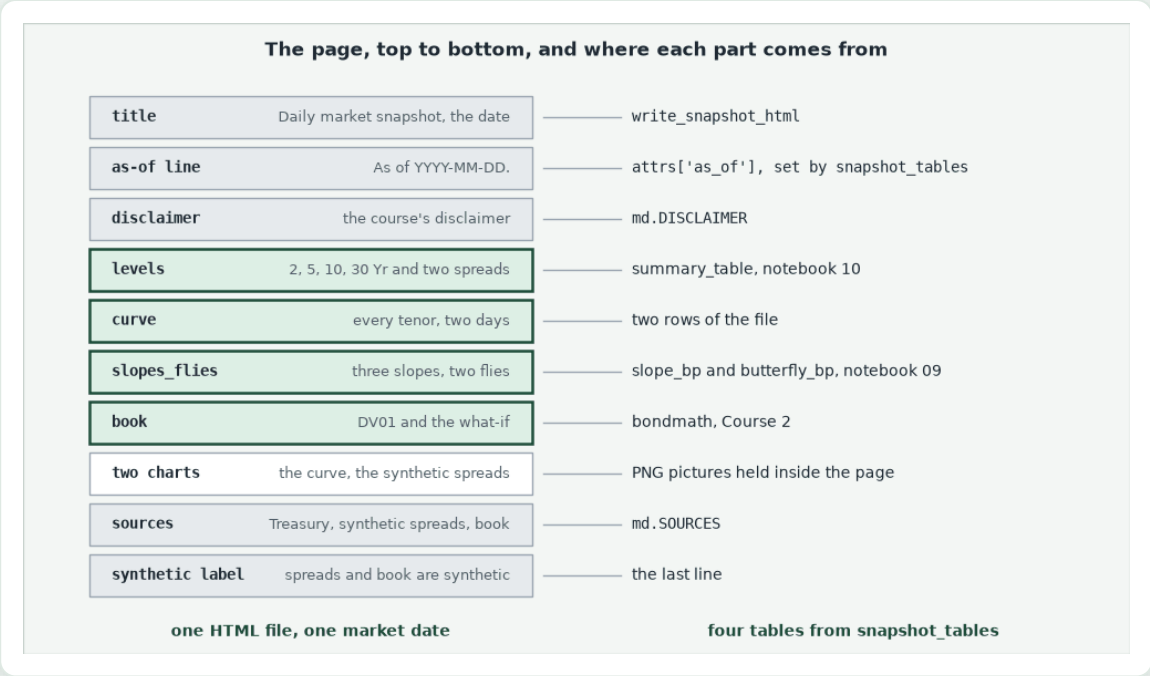
Imports and the Market

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date',
                      float_precision='round_trip')
market = treasury.join(spreads, validate='one_to_one')
print(f'{len(market):,} rows, {market.index[0]:%Y-%m-%d} to {market.index[-1]:%Y-%m-%d}, '
      f'{market.shape[1]} columns')
2,940 rows, 2015-01-02 to 2026-10-02, 16 columns
```

"validate='one_to_one'" stops the join if a date repeats on either side, as notebook 12 checked its joins.

Today the course's measures become one page, for any date, from one command.

14.1 One page, one date, one rule



The note's page, top to bottom, each part tagged with where it comes from. Three weeks of "marketdata" feed it, and Course 2's "bondmath" prices the book. Every line on it obeys notebook 08's rule.

Describes	Never
was, rose, fell, over a day, a week, a month	will, expect, should
a z-score over 252 rows, today included	stretched, due to revert
the share of the window at or below today	cheap, rich, a signal
what the book would do if the curve moved so, labelled a what-if	what the book will do

The September Book

```
book = pd.read_csv('holdings.csv')
SNAPSHOT_UNITS = {'2 Yr': 'pct', '5 Yr': 'pct', '10 Yr': 'pct', '30 Yr': 'pct',
                  'ig_spread_bp': 'bp', 'hy_spread_bp': 'bp'}
print(len(book), 'bonds as of', book['as_of_date'].iloc[0])
200 bonds as of 2026-09-30
```

One date, four tables, two charts, and one rule: every number says where something was, never where it goes.

14.2 snapshot_tables, docstring first

"snapshot_tables(market, book, as_of, units)" returns four tables for one date, each from rows on or before it.

"known = market.loc[:as_of]", notebook 10's cut.

"levels", notebook 10's "summary_table" of the columns in "units".

"curve", every tenor on "as_of" and on the row before, with the change in bp.

"slopes_flies", notebook 09's slopes and flies, summarised the same way.

"book", the September book's DV01 and a what-if, from a helper that 14.3 reads.

"*legs" hands a tuple over as separate arguments, the short leg, the belly and the long leg.

```
%%writefile -a marketdata.py
def snapshot_tables(market: pd.DataFrame, book: pd.DataFrame, as_of, units: dict[str, str],
                    window: int = 252) -> dict[str, pd.DataFrame]:
    """The tables of a one-page daily market snapshot for `as_of`, from rows on or before it only.
    Units: yields in percent, spreads, slopes, flies, and changes in bp, dollars in the book table.
    Convention:
    - levels: summary_table of the columns named in `units`;
    - curve: each tenor's yield on as_of and on the previous observation, and the change in bp;
    - slopes_flies: summary_table of 2s10s, 5s30s, 3m10y, 2s5s10s, and 5s10s30s (all in bp);
    - book: the book's position DV01 (bondmath.dv01 at the book's own as_of_date) and the mechanical change
      in market value if each bond's yield moved by the day's change in the par curve, interpolated at its
      maturity with bondmath.interpolate_yield; by sector, by rating bucket, and in total. A what-if.
    Each table's attrs["as_of"] holds the date as YYYY-MM-DD, for the writers.
    Raises ValueError if as_of is not a market date (a date with a row in `market`).
    """
    _check_index(market.index, "market")
    as_of = pd.Timestamp(as_of)
    if as_of not in market.index:
        raise ValueError(f"{as_of.date()} is not a market date: market has no row for it")
    known = market.loc[:as_of]
    levels = summary_table(known[list(units)], as_of, units, window)
    tenors = [column for column in known.columns if column in TENOR_YEARS]
    previous_date = known.index[-2]
    curve = pd.DataFrame({"years": [TENOR_YEARS[t] for t in tenors],
                          "previous_pct": known.loc[previous_date, tenors].to_numpy(),
                          "as_of_pct": known.loc[as_of, tenors].to_numpy()}, index=pd.Index(tenors, name="tenor"))
    curve["chg_bp"] = (curve["as_of_pct"] - curve["previous_pct"]) * 100
    curve = curve.dropna().sort_values("years")
    shapes = pd.DataFrame({name: slope_bp(known, short, long) for name, (short, long) in SLOPES.items()}
                          | {name: butterfly_bp(known, *legs) for name, legs in FLIES.items()})
    slopes_flies = summary_table(shapes, as_of, {name: "bp" for name in shapes.columns}, window)
    tables = {"levels": levels, "curve": curve, "slopes_flies": slopes_flies, "book": _book_section(book, curve)}
    for table in tables.values():
        table.attrs["as_of"] = as_of.strftime("%Y-%m-%d")
    return tables
```

The constants and the book helper

Today's imports sit where today's code starts, as in notebook 13. The leading underscore marks "_book_section" as a helper, as notebook 03's "_check_index". "book.itertuples()" gives each row as an object, so "bond.coupon" is that row's coupon. ".agg(sums)" hands the dictionary "sums" over as named arguments, one per column of the result.

"reset_index(names='group')" turns the row labels into a column named "group". "rating.rstrip('+-')" cuts "+" and "-" off the end, as Course 2's ".str.rstrip", so "BBB+" becomes "BBB".

```
def _book_section(book: pd.DataFrame, curve: pd.DataFrame) -> pd.DataFrame:
    """The book's position DV01 and the mechanical change in market value from the day's curve change, by
    sector and by rating bucket, then the total (a helper for snapshot_tables, not part of the API)."""
    settlement_dates = book["as_of_date"].unique()
    if len(settlement_dates) != 1:
        raise ValueError(f"the book must have one as_of_date, not {len(settlement_dates)}")
    settlement = date.fromisoformat(settlement_dates[0])
    tenors_years = curve["years"].tolist()
    changes_bp = curve["chg_bp"].tolist()
    rows = []
    for bond in book.itertuples():
        maturity = date.fromisoformat(bond.maturity)
        dv01_usd = bondmath.dv01(settlement, maturity, bond.coupon, bond.yield_pct) * bond.par_held / 100
        shift_bp = bondmath.interpolate_yield(bondmath.years_to_maturity(settlement, maturity), tenors_years, changes_bp)
        before = bondmath.dirty_price(settlement, maturity, bond.coupon, bond.yield_pct)
        after = bondmath.dirty_price(settlement, maturity, bond.coupon, bond.yield_pct + shift_bp / 100)
        rows.append({"sector": bond.sector, "bucket": bond.rating.rstrip("+-"), "face_usd": bond.par_held,
                     "market_value_usd": bond.market_value, "dv01_usd": dv01_usd,
                     "mv_change_usd": (after - before) * bond.par_held / 100})
    positions = pd.DataFrame(rows)
```

2 October: the levels, against Excel

Each table carries "attrs", a dictionary of facts about the table itself. "attrs['as_of']" holds the date, and the writers read it from there.

```
importlib.reload(md)
AS_OF = '2026-10-02'
tables = md.snapshot_tables(market, book, AS_OF, SNAPSHOT_UNITS)
print(list(tables), 'as of', tables['levels'].attrs['as_of'])
['levels', 'curve', 'slopes_flies', 'book'] as of 2026-10-02
```

	unit	level	chg_1d_bp	chg_1w_bp	chg_1m_bp	zscore	pct_rank
2 Yr	pct	4.83	5.00	2.00	44.00	2.51	98.02
5 Yr	pct	5.06	5.00	8.00	52.00	2.80	99.60
10 Yr	pct	5.28	4.00	11.00	49.00	3.08	99.60
30 Yr	pct	5.63	2.00	14.00	36.00	3.08	99.60
ig_spread_bp	bp	97.00	-1.00	1.10	1.90	0.22	61.11
hy_spread_bp	bp	328.70	-1.30	11.20	0.10	0.90	73.81

Against Excel's Reference Row

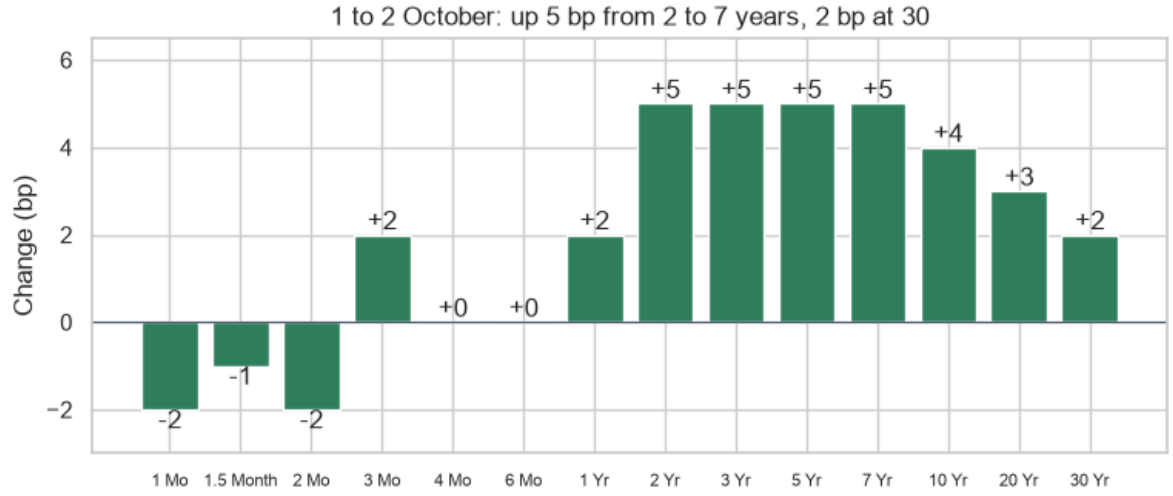
```
10 Yr: z-score gap 4.6e-13, rank gap 0.0
2s10s: z-score gap 1.1e-15, rank gap 0.0
```

The 10 Yr closed at 5.28 percent, 4 bp up on the day and 49 bp on the month. Its z-score is 3.08, with 99.6 percent of its last 252 rows at or below it.

Excel side by side. The 10 Yr's last 252 values in A1:A252, today in A252.

Job	Python	Excel	Result
Z-score	`tables['levels'].loc['10 Yr', 'zscore']`	`=STANDARDIZE(A252,AVERAGE(A1:A252),STDEV.S(A1:A252))`	3.075234
Percentile rank	`tables['levels'].loc['10 Yr', 'pct_rank']`	`=COUNTIF(A1:A252,"<="&A252)/COUNT(A1:A252)*100`	99.603175

The curve's move, and the slopes and flies



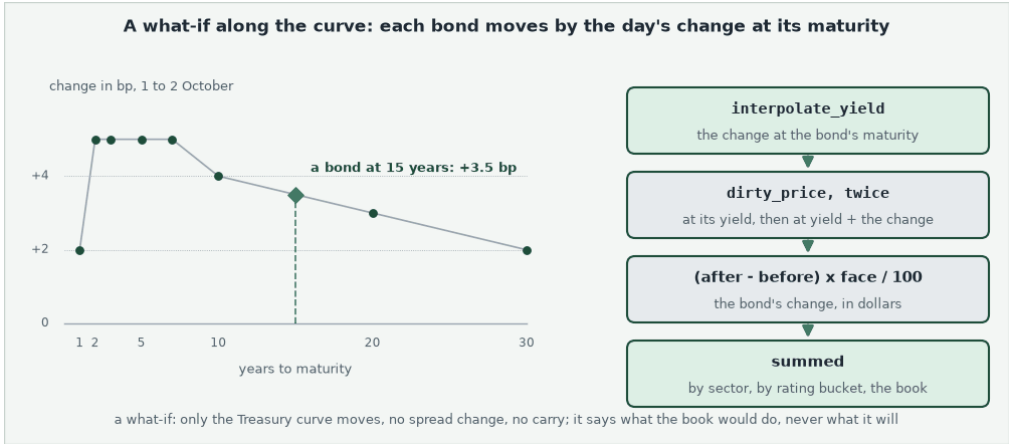
Slopes and Flies

	unit	level	chg_1d_bp	chg_1w_bp	chg_1m_bp	zscore	pct_rank
2s10s	bp	45.00	-1.00	9.00	5.00	-0.51	28.17
5s30s	bp	57.00	-3.00	6.00	-16.00	-2.07	5.95
3m10y	bp	109.00	2.00	16.00	22.00	2.07	100.00
2s5s10s	bp	1.00	1.00	3.00	11.00	2.70	98.81
5s10s30s	bp	-13.00	1.00	0.00	10.00	2.58	100.00

2s10s is 45 bp, with 28.2 percent of its year at or below it, and 5s30s 57 bp at 5.95 percent.

One call builds three of the page's tables for 2 October, and the 10 Yr's z-score ties to Excel's within 4.6e-13.

14.3 The book section: DV01 and a what-if



The fourth table moves each bond's yield by the day's curve change at its maturity, Course 2's "interpolate_yield", and prices it again. It is a what-if, not a forecast. Only the Treasury curve moves, with no spread change and no carry. The prices are 30 September's and the move is 2 October's. Course 2's notebook 14 moved every yield by the same amount, and here the move differs by maturity.

Control Totals

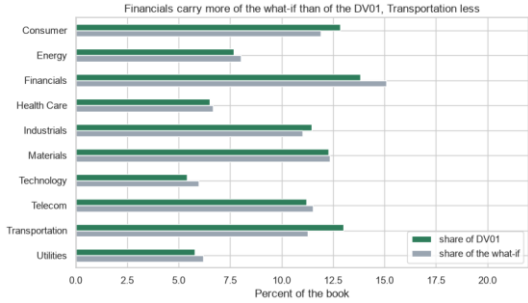
```
numbers = ['bonds', 'face_usd', 'market_value_usd', 'dv01_usd', 'mv_change_usd']
book_table = tables['book']
for by in ['sector', 'rating bucket']:
    gap = book_table[book_table['by'] == by][numbers].sum() - total[numbers].astype(float)
    assert (gap.abs() < 0.005).all(), by
print('sectors and rating buckets each add to the book on all five columns')
sectors and rating buckets each add to the book on all five columns
```

DV01 by sector and rating bucket

by	group	bonds	face_usd	market_value_usd	dv01_usd
sector	Consumer	26	56,250,000.00	56,629,597.50	39,489.93
sector	Energy	17	39,250,000.00	38,830,005.00	23,542.53
sector	Financials	26	69,250,000.00	68,219,672.50	42,434.63
sector	Health Care	20	39,000,000.00	38,908,187.50	19,941.82
sector	Industrials	20	50,750,000.00	52,540,745.00	35,177.59
sector	Materials	26	70,500,000.00	71,701,617.50	37,739.11
sector	Technology	12	30,000,000.00	30,311,760.00	16,572.55
sector	Telecom	21	54,750,000.00	55,848,270.00	34,384.38
sector	Transportation	20	50,000,000.00	48,822,897.50	39,971.07
sector	Utilities	12	29,250,000.00	29,006,462.50	17,742.51
rating bucket	A	56	147,500,000.00	148,505,040.00	95,015.04
rating bucket	AA	8	12,750,000.00	13,048,305.00	5,402.62
rating bucket	AAA	2	7,750,000.00	8,200,605.00	4,679.14
rating bucket	B	18	44,500,000.00	43,595,535.00	23,095.54
rating bucket	BB	49	111,750,000.00	113,754,240.00	63,536.72
rating bucket	BBB	62	151,750,000.00	151,163,415.00	109,668.71
rating bucket	CCC	5	13,000,000.00	12,552,075.00	5,598.34
total	book	200	489,000,000.00	490,819,215.00	306,996.11

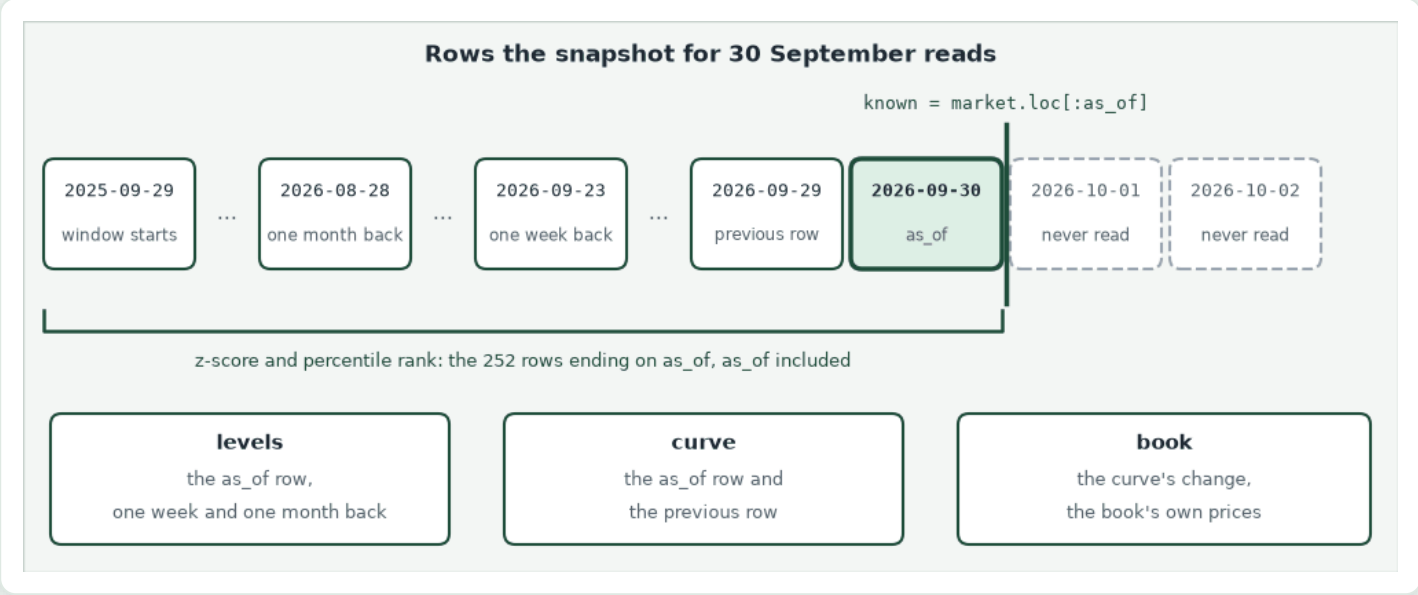
```
money = ['face_usd', 'market_value_usd', 'dv01_usd']
dv01_table = tables['book'].drop(columns='mv_change_usd')
dv01_table.style.format('{:,.2f}', subset=money).hide(axis='index')
```

```
by_sector = tables['book'][tables['book']['by'] == 'sector'].set_index('group')
dv01_share = by_sector['dv01_usd'] / total['dv01_usd'] * 100
what_if_share = by_sector['mv_change_usd'] / total['mv_change_usd'] * 100
shares = pd.DataFrame({'share of DV01': dv01_share, 'share of the what-if': what_if_share})
shares.loc[['Financials', 'Transportation']].round(2)
```



14.4 Only rows on or before the date

The note's last line: a page for 30 September shows 30 September.



Notebook 10's picture showed the cut. This one shows which rows behind it each table reads.

A Date With No Row

```
md.snapshot_tables(market, book, '2026-09-07', SNAPSHOT_UNITS)
```

```
ValueError: 2026-09-07 is not a market date: market has no row for it
```

30 September, Cut and Uncut

```
full = md.snapshot_tables(market, book, '2026-09-30', SNAPSHOT_UNITS)
cut_at = md.snapshot_tables(market.loc[:'2026-09-30'], book, '2026-09-30', SNAPSHOT_UNITS)
for name in full:
    pd.testing.assert_frame_equal(full[name], cut_at[name], check_exact=True)
print(f"four tables equal; the 10 Yr on 30 September: {full['levels'].loc['10 Yr', 'level']:.2f}")
four tables equal; the 10 Yr on 30 September: 5.29
```

Two tests

"snapshot_inputs" loads the market and the book as this notebook does. The first test asks for 30 September from the full file, the second for Labor Day.

```
%%writefile -a test_marketdata.py
from matplotlib.figure import Figure  # a chart without a screen, from day 14
SNAPSHOT_UNITS = {"2 Yr": "pct", "5 Yr": "pct", "10 Yr": "pct", "30 Yr": "pct", "ig_spread_bp": "bp", "hy_spread_bp": "bp"}
def snapshot_inputs() -> tuple[pd.DataFrame, pd.DataFrame]:
    spreads = pd.read_csv(HERE / "course3_synthetic_spreads.csv", parse_dates=["date"], index_col="date")
    market = read_treasury().join(spreads, validate="one_to_one")
    return market, pd.read_csv(HERE / "holdings.csv")
def test_snapshot_uses_as_of_not_last_row():
    market, book = snapshot_inputs()
    tables = md.snapshot_tables(market, book, "2026-09-30", SNAPSHOT_UNITS)
    assert tables["levels"].loc["10 Yr", "level"] == 5.29
    assert tables["curve"].loc["10 Yr", "as_of_pct"] == 5.29
    assert tables["curve"].loc["10 Yr", "previous_pct"] == market.loc["2026-09-29", "10 Yr"]
    assert tables["levels"].attrs["as_of"] == "2026-09-30"
    total = tables["book"].set_index("by").loc["total"]
    assert total["bonds"] == 200
def test_snapshot_rejects_non_market_date():
    market, book = snapshot_inputs()
    with pytest.raises(ValueError):
        md.snapshot_tables(market, book, "2026-09-07", SNAPSHOT_UNITS)
```

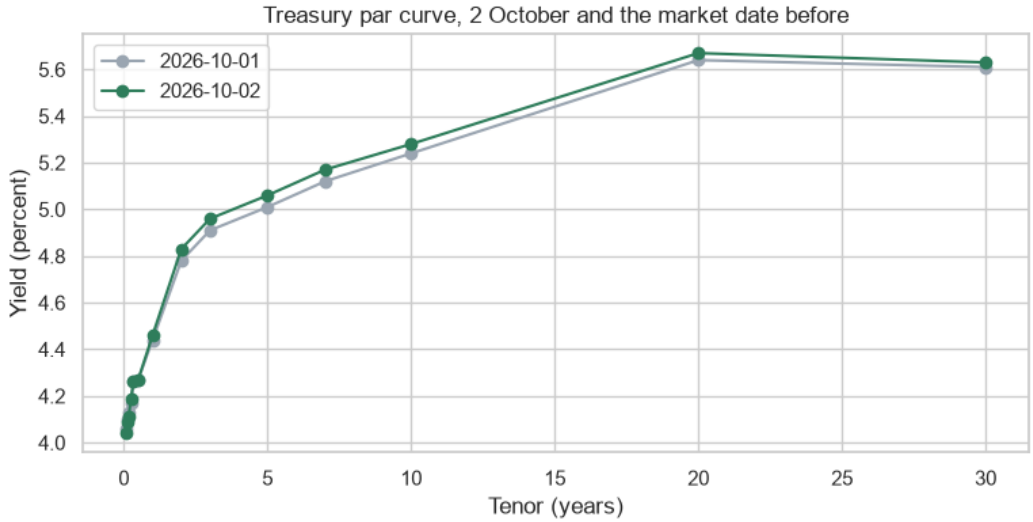
```
.....................................................   [100%]
53 passed in 0.50s
```

The snapshot refuses a date the market never had and never reads past its date, and two tests now hold both.

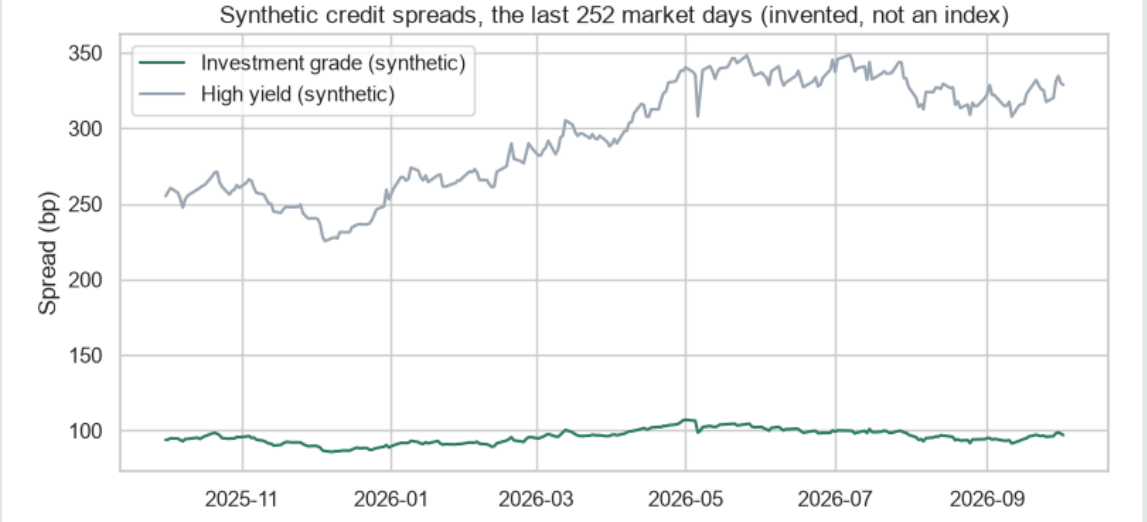
14.5 Two charts for the page

The note asks for a web page, and the page needs its two charts.

The Curve on Both Days



The Synthetic Spreads, the Last 252 Days



The page takes the chart objects themselves, "curve_figure" and "spread_figure", which matplotlib calls a "Figure".

```
curve_figure, ax = plt.subplots(figsize=(9, 4))
```

write_snapshot_html

It writes one self-contained page, with every picture inside the file. "table.to_html(float_format="{:,.2f}".format)" writes a table as HTML, the string's "format" used as a function. "html.escape" makes text safe for HTML, so "&" is written "&".

Each chart goes into "io.BytesIO()", a file held in memory, as a PNG. "base64" turns the PNG's bytes into plain text, so the picture sits inside the page as a data URI.

```
%%writefile -a marketdata.py
def write_snapshot_html(tables: dict[str, pd.DataFrame], figures: dict[str, Figure], path: Path, title: str) -> Path:
    """Write the snapshot as one self-contained HTML file and return its path.
    Units: as in the tables (numbers shown to 2 decimals). Convention: the title, the as-of date (from the
    tables' attrs, set by snapshot_tables), the disclaimer, each table as DataFrame.to_html, each chart as a
    PNG embedded in base64 (no separate image files), the data sources, and the synthetic label. Describes the
    data; it contains no forecast and no recommendation. Uses no template library.
    Raises ValueError if the tables carry no as-of date.
    """
    as_of = tables["levels"].attrs.get("as_of")
    if as_of is None:
        raise ValueError("the tables carry no as-of date; build them with snapshot_tables")
    parts = [f"<h1>{html.escape(title)}</h1>", f"<p>As of {as_of}.</p>",
             f"<p><em>{html.escape(DISCLAIMER)}</em></p>"]
    for name, table in tables.items():
        parts.append(f"<h2>{html.escape(name)}</h2>")
        parts.append(table.to_html(float_format="{:,.2f}".format))
    for name, figure in figures.items():
        buffer = io.BytesIO()
        figure.savefig(buffer, format="png", dpi=100)
        encoded = base64.b64encode(buffer.getvalue()).decode("ascii")
        parts.append(f"<h2>{html.escape(name)}</h2>")
        parts.append(f'<img alt="{html.escape(name)}" src="{encoded}">')
    parts.append(f"<p>{html.escape(SOURCES)}</p>")
    parts.append("<p>The credit spreads and the book are synthetic.</p>")
    page = ("<!DOCTYPE html>\n<html lang=\"en\">\n<head>\n<meta charset=\"utf-8\">\n"
            f"<title>{html.escape(title)}</title>\n</head>\n<body>\n" + "\n".join(parts) + "\n</body>\n</html>\n")
    path = Path(path)
    path.write_text(page, encoding="utf-8", newline="\n")
    return path
```

Write the page, then check it as text

```
importlib.reload(md)
Path('snapshots').mkdir(exist_ok=True)
figures = {'Treasury curve': curve_figure, 'Synthetic credit spreads': spread_figure}
html_path = md.write_snapshot_html(tables, figures, Path('snapshots') / f'snapshot_{AS_OF}.html',
                                   f'Daily market snapshot, {AS_OF}')
print(html_path.name)
snapshot_2026-10-02.html
```

"HTML(text)" marks text as a web page, and "display" shows it here, as a browser would.

The same file opens in your browser with PowerShell's "Invoke-Item", which opens a file with its own program (Bring It Home).

```
re.findall(r'<h2>(.+?)</h2>', page)
['levels',
 'curve',
 'slopes_flies',
 'book',
 'Treasury curve',
 'Synthetic credit spreads']
```

```
checks = {'the date line': f'<p>As of {AS_OF}.</p>' in page,
          'the disclaimer': md.DISCLAIMER in page,
          'the sources': 'Daily Treasury Par Yield Curve Rates' in page,
          'the synthetic label': 'The credit spreads and the book are synthetic.' in page,
          'four tables': page.count('<table') == 4,
          'two pictures': page.count('') == 2,
          'no home folder or work folder': str(Path.home()) not in page and work.name not in page}
assert all(checks.values()), checks
print(f'{sum(checks.values())} of {len(checks)} checks pass')
7 of 7 checks pass
```

One function writes one self-contained file, title to sources, and seven plain checks prove what is on it.

14.6 The workbook, opened in Excel

The note asks for a workbook to check in Excel. "write_snapshot_excel" puts an "about" sheet first, with the date and the sources, then one sheet per table. It uses "pd.ExcelWriter", as Course 2's Portfolio Aggregation and Scenarios did.

```
%%writefile -a marketdata.py
def write_snapshot_excel(tables: dict[str, pd.DataFrame], path: Path) -> Path:
    """Write the snapshot tables to one Excel workbook and return its path.
    Units: as in the tables. Convention: pd.ExcelWriter with openpyxl. The first sheet, about, holds the as-of
    date (from the tables' attrs, set by snapshot_tables), the data sources, the synthetic label, and the
    disclaimer, so the workbook describes itself as the HTML page does. Then one sheet per table, named by its
    key, holding the table with its index.
    Raises ValueError if the tables carry no as-of date.
    """
    as_of = tables["levels"].attrs.get("as_of")
    if as_of is None:
        raise ValueError("the tables carry no as-of date; build them with snapshot_tables")
    about = pd.DataFrame({"value": [as_of, SOURCES, "The credit spreads and the book are synthetic.", DISCLAIMER]},
                         index=pd.Index(["as_of", "sources", "synthetic", "disclaimer"], name="item"))
    path = Path(path)
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        about.to_excel(writer, sheet_name="about")
        for name, table in tables.items():
            table.to_excel(writer, sheet_name=name)
    return path
```

Read It Back

```
importlib.reload(md)
xlsx_path = md.write_snapshot_excel(tables, Path('snapshots') / f'snapshot_{AS_OF}.xlsx')
back = pd.read_excel(xlsx_path, sheet_name=None, index_col=0)
print(list(back), 'as of', back['about'].loc['as_of', 'value'])
print({name: len(table) for name, table in back.items()})
```

```
['about', 'levels', 'curve', 'slopes_flies', 'book'] as of 2026-10-02
{'about': 4, 'levels': 6, 'curve': 14, 'slopes_flies': 5, 'book': 18}
```

```
pd.testing.assert_frame_equal(back['levels'], tables['levels'], check_exact=False, rtol=1e-12,
                              check_dtype=False)
print('levels: equal')
```

```
levels: equal
```

What Excel shows, and two more tests

Excel side by side. The workbook as written, opened in Excel 16.0 (build 20430) read-only and closed unsaved. In the book sheet the index is column A, "by" is B, the DV01 G, and the what-if H.

Job	Python	Excel	Result
The sheets	`list(back)`	`the tabs`	`about, levels, curve, slopes_flies, book`
The curve's change	`chg_bp`	`=(D2-C2)*100 in F, filled down`	`within 8.9e-16 bp of column E`
DV01 by sector	`dv01_usd`	`=SUMIFS(G2:G19,B2:B19,"sector")`	`306996.110402`
DV01 by rating	`dv01_usd`	`=SUMIFS(G2:G19,B2:B19,"rating bucket")`	`306996.110402`

```
%%writefile -a test_marketdata.py
def test_html_contains_as_of_and_disclaimer(tmp_path):
    market, book = snapshot_inputs()
    tables = md.snapshot_tables(market, book, "2026-10-02", SNAPSHOT_UNITS)
    figure = Figure(figsize=(4, 3))
    axes = figure.subplots()
    axes.plot(tables["curve"]["years"], tables["curve"]["as_of_pct"])
    path = md.write_snapshot_html(tables, {"curve": figure}, tmp_path / "snapshot.html", "Daily market snapshot")
    text = path.read_text(encoding="utf-8")
    assert "As of 2026-10-02." in text
    assert md.DISCLAIMER in text
    assert "synthetic" in text and "" in text
    assert text.count("<table") == 4
def test_excel_round_trip(tmp_path):
    market, book = snapshot_inputs()
    tables = md.snapshot_tables(market, book, "2026-10-02", SNAPSHOT_UNITS)
    path = md.write_snapshot_excel(tables, tmp_path / "snapshot.xlsx")
    back = pd.read_excel(path, sheet_name=None, index_col=0)
    assert list(back) == ["about", *tables]
    assert back["about"].loc["as_of", "value"] == "2026-10-02"
    assert back["about"].loc["disclaimer", "value"] == md.DISCLAIMER
    for name, table in tables.items():
        assert len(back[name]) == len(table)
    levels = back["levels"]
    levels.index.name = tables["levels"].index.name
    pd.testing.assert_frame_equal(levels, tables["levels"], check_exact=False, rtol=1e-12, check_dtype=False)
```

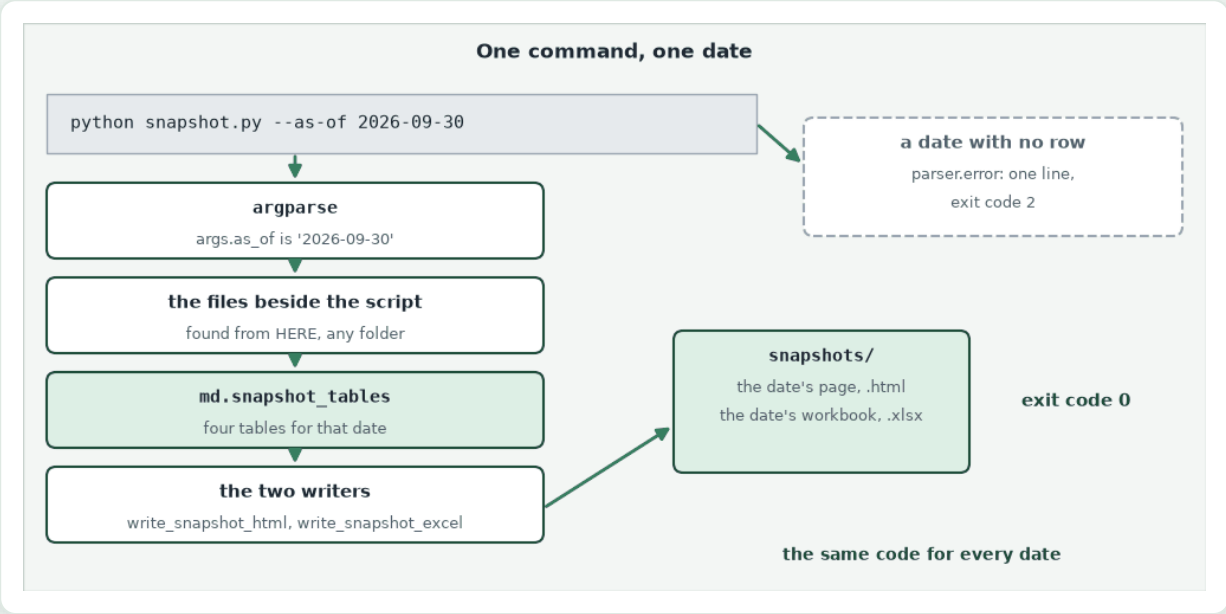
The page test draws one small chart with "Figure(figsize=(4, 3))", a chart made with no screen, as a script needs.

```
.....................................................   [100%]
55 passed in 0.50s
```

The workbook opens in Excel with its date on the first sheet, and Excel's own "SUMIFS" gives the book's DV01 back to the sixth decimal.

14.7 snapshot.py: one command, any date

A script is a ".py" file started from PowerShell, "python snapshot.py", not imported.



"argparse" reads the command line, and "--as-of 2026-09-30" arrives as "args.as_of", the hyphen an underscore.

"HERE = Path(__file__).resolve().parent" is the script's own folder, so it runs from any folder.

"if __name__ == "__main__":" runs "main()" only when the file is started as a script, as in Course 1's From Notebook to Module.

An exit code says how a run went, 0 for success and anything else for not, and "run.returncode" holds it.

The script's main, and its help

```
%%writefile snapshot.py
...
def main() -> None:
    parser = argparse.ArgumentParser(description="Write the daily market snapshot as HTML and Excel.")
    parser.add_argument("--as-of", help="the market date, YYYY-MM-DD (default: the last date in the market data)")
    args = parser.parse_args()
    market = load_market()
    book = pd.read_csv(HERE / "holdings.csv")
    as_of = args.as_of or f"{market.index[-1]:%Y-%m-%d}"
    try:
        tables = md.snapshot_tables(market, book, as_of, UNITS)
    except ValueError as error:
        # a one-line message and exit code 2, with no traceback: a traceback would print this file's full path
        parser.error(str(error))
    # the date as the tables carry it, YYYY-MM-DD, names both files
    as_of = tables["levels"].attrs["as_of"]
    out = HERE / "snapshots"
    out.mkdir(exist_ok=True)
    md.write_snapshot_html(tables, charts(tables, market), out / f"snapshot_{as_of}.html", f"Daily market snapshot, {as_of}")
    md.write_snapshot_excel(tables, out / f"snapshot_{as_of}.xlsx")
    print(f"snapshot for {as_of}: wrote snapshots/snapshot_{as_of}.html and snapshots/snapshot_{as_of}.xlsx")
if __name__ == "__main__":
    main()
```

"args.as_of or ..." takes the first value that is not empty, so a run with no "--as-of" gets the file's last date. "autofmt_xdate()" tilts the spread chart's date labels so they do not overlap.

Its Help

```
usage: snapshot.py [-h] [--as-of AS_OF]
Write the daily market snapshot as HTML and Excel.
options:
  -h, --help   show this help message and exit
  --as-of AS_OF  the market date, YYYY-MM-DD (default: the last date in the
   market data)
```

A date with no row, and rerun, do not redo

A Date With No Row

```
bad_run = subprocess.run([sys.executable, 'snapshot.py', '--as-of', '2026-09-07'], cwd=work,
                         capture_output=True, text=True)
print('exit code', bad_run.returncode)
print(bad_run.stderr)
exit code 2
usage: snapshot.py [-h] [--as-of AS_OF]
snapshot.py: error: 2026-09-07 is not a market date: market has no row for it
```

One line and exit code 2, with no traceback, which would print the script's full path.

Rerun, Do Not Redo

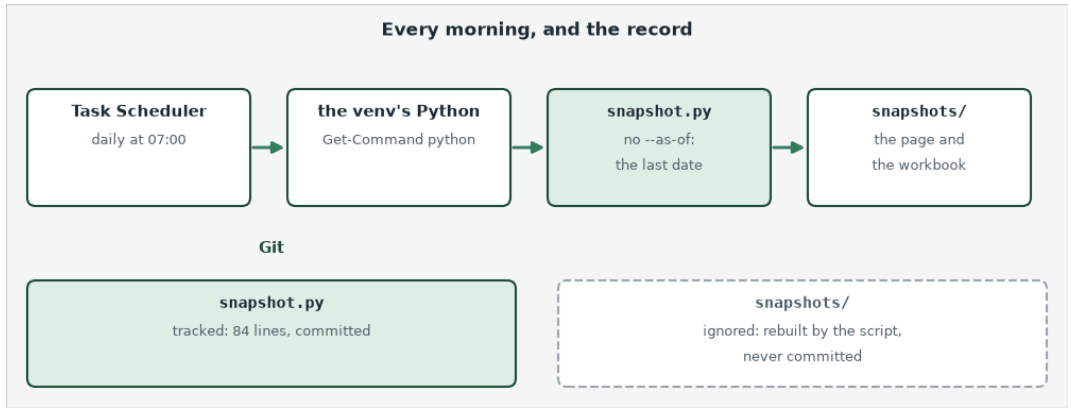
```
two_days = {day: pd.read_excel(Path('snapshots') / f'snapshot_{day}.xlsx', sheet_name='levels',
                               index_col=0)[['level', 'zscore', 'pct_rank']]
            for day in ['2026-09-30', '2026-10-02']}
side_by_side = pd.concat(two_days, axis=1)
side_by_side.round(2)
```

The 10 Yr shows 5.29 on 30 September and 5.28 on 2 October.

	2026-09-30			2026-10-02		
	level	zscore	pct_rank	level	zscore	pct_rank
2 Yr	4.88	2.73	99.21	4.83	2.51	98.02
5 Yr	5.09	2.99	100.00	5.06	2.80	99.60
10 Yr	5.29	3.25	100.00	5.28	3.08	99.60
30 Yr	5.64	3.26	100.00	5.63	3.08	99.60
ig_spread_bp	98.80	0.61	73.02	97.00	0.22	61.11
hy_spread_bp	334.70	1.09	82.94	328.70	0.90	73.81

One command writes the page and the workbook for any market date, and the code did not change between the two.

14.8 Every morning, and the record



What Git Sees

```
!git -C "{work}" status --short
 M marketdata.py
 M test_marketdata.py
?? snapshot.py
?? snapshots/
```

Windows' Task Scheduler runs a command at a set time, and "schtasks" is its command-line tool. These lines are text for your own machine, never run here. At work, your firm's policy decides whether a scheduled task may run.

```
# the full path of the venv's Python (the course's .venv active)
$python = (Get-Command python).Source
# /TN names the task, /SC DAILY and /ST set when, /TR is the command
schtasks /Create /TN "pfi_daily_snapshot" /SC DAILY /ST 07:00 /TR "'$python' '$HOME\projects\my_bondmath\snapshot.py'"
# look at it, run it once now, delete it
schtasks /Query /TN "pfi_daily_snapshot"
schtasks /Run /TN "pfi_daily_snapshot"
schtasks /Delete /TN "pfi_daily_snapshot" /F
```

"schtasks /Create /?" says a path with spaces needs a second set of quotes inside the first, and single quotes will do. The course's files stop on 2 October, so every run writes the same page until the data is refreshed, the capstone's job.

Keep the pages out, commit the day

Keep the Pages Out

```
%%writefile -a .gitignore
# the snapshot outputs: rebuilt by python snapshot.py, never committed
snapshots/
```

```
!git -C "{work}" status --short --ignored
 M .gitignore
 M marketdata.py
 M test_marketdata.py
?? snapshot.py
!! __pycache__/
!! bondmath.py
!! course3_excel_reference.csv
!! course3_fred_format_dgs10.json
!! course3_fred_format_dgs2.json
!! course3_synthetic_spreads.csv
!! holdings.csv
!! ratings.csv
!! snapshots/
!! trade_blotter.csv
!! treasury_par_yields.csv
```

"snapshots/" is ignored now. The data files and "bondmath.py" show "!!" too, from the work folder's own ignore list.

A second "-m" adds a body under the subject. "git log --stat -1" shows the last commit with each file's count of changed lines.

```
subject = ('Notebook 14: snapshot_tables, write_snapshot_html, write_snapshot_excel, '
           'and snapshot.py')
body = 'The pages are rebuilt by python snapshot.py, so snapshots/ is ignored.'
!git -C "{work}" diff --stat
!git -C "{work}" add .gitignore marketdata.py test_marketdata.py snapshot.py
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" log --stat -1
 .gitignore    |   2 +
 marketdata.py   | 157 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py |  62 ++++++++++++++++++++++
 3 files changed, 221 insertions(+)
commit d20a0477c2524154d29765e640ced5799a384d1c
Author: Course Student <student@example.com>
Date:   Wed Sep 30 09:00:00 2026 +0000
    Notebook 14: snapshot_tables, write_snapshot_html, write_snapshot_excel, and snapshot.py
    The pages are rebuilt by python snapshot.py, so snapshots/ is ignored.
 .gitignore    |   2 +
 marketdata.py   | 157 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 snapshot.py   |  84 ++++++++++++++++++++++++++++++
 test_marketdata.py |  62 ++++++++++++++++++++++
 4 files changed, 305 insertions(+)
```

Task Scheduler can run the command every morning, and Git keeps the 84 lines that write the pages and none of the pages.

Bring it home, and what you can now do

Copy the work folder's module, tests and script into your own "my_bondmath" folder, test them there, run the script, and commit. The copy replaces your files, so anything of your own must be in the work folder first. The script reads the synthetic spreads, so copy them too; the Treasury file and the book are there since notebooks 04 and 11. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_14_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\snapshot.py" .
Copy-Item "$work\course3_synthetic_spreads.csv" .
Add-Content .gitignore "snapshots/"
python -m pytest -q test_marketdata.py
python snapshot.py
Invoke-Item snapshots\snapshot_2026-10-02.html
git status --short
git add .gitignore marketdata.py test_marketdata.py snapshot.py
git commit -m "Notebook 14: snapshot_tables, write_snapshot_html, write_snapshot_excel, and snapshot.py"
```

pytest shows 55 passed, and the page opens in your browser. In Colab, download the three files and the "snapshots" folder from the work folder under "/tmp" instead. The work folder stays in your temp folder; the system clears it in time.

You Can Now

Build a one-page snapshot's tables for any market date from rows on or before it only.

Put the book's DV01 by sector and rating beside a what-if of the day's curve move, labelled as one.

Write a self-contained HTML page and an Excel workbook, and check both as data.

Prove a date-driven report reads nothing after its date.

Turn notebook code into a script with "argparse", run it for any date, and read its exit code.

Schedule a script with Task Scheduler and keep its outputs out of Git.

Predict first: the what-if, and an early date

Predict 1

The book's DV01 is 306,996.11 dollars per bp, and the curve rose 2 to 5 bp from the 1 Yr out. Move each bond's yield by the change at its maturity and price it again: by how many dollars does the book's value change? Write down a number.

```
total = tables['book'].iloc[-1]
print(f"what-if change in value: {total['mv_change_usd']:,.2f} dollars")
average_move_bp = -total['mv_change_usd'] / total['dv01_usd']
print(f'{average_move_bp:.2f} bp of DV01')
```

Predict 2

2015-06-01 has a row, the file's 104th. The z-score and the percentile rank use 252 rows. Its 10 Yr row shows:

A) both from the 104 rows so far

B) the level and the changes, with the z-score and the rank empty

C) the z-score empty, the rank from the rows so far

```
early = md.snapshot_tables(market, book, '2015-06-01', SNAPSHOT_UNITS)
early['levels'].loc['10 Yr']
```

Predict first: Excel's display, and the files

Predict 3

The writer sets no number format and no column width. In Excel, the book sheet's total face, 489,000,000, shows as:

A) 489000000

B) 4.89E+08

C) "########"

```
importlib.reload(md)
xlsx_path = md.write_snapshot_excel(tables, Path('snapshots') / f'snapshot_{AS_OF}.xlsx')
```

Predict 4

14.5 and 14.6 left two files for 2 October in "snapshots". Run the script for 2 October, then for 30 September. How many files are in "snapshots" then? Write down a number.

```
for as_of in ['2026-10-02', '2026-09-30']:
    run = subprocess.run([sys.executable, 'snapshot.py', '--as-of', as_of], cwd=work,
                         capture_output=True, text=True)
    assert run.returncode == 0, run.stderr
    print(run.stdout.strip())
sorted(path.name for path in Path('snapshots').glob('*'))
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "today = market.iloc[-1]" reads the file's last row, not as_of's, though the 10 Yr on 2026-09-30 was 5.29 percent.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
Closes and moves	10 Yr 5.28 percent, up 4 bp on the day and 49 bp on the month; synthetic HY 328.7 bp, down 1.3 bp
Each in its own year	10 Yr z-score 3.08, 99.6 percent at or below; 2s10s 45 bp, 28.2 percent
Page and workbook	snapshot_2026-10-02.html and .xlsx, both checked
Any date	python snapshot.py --as-of, run for two dates
Every morning, the record	schtasks as text; snapshot.py committed, snapshots/ ignored

Be proud: a desk report built from functions you wrote and tested, for any date, with nothing read from the future.

Next: the Course 3 Capstone, where a vendor's extract is cleaned, proved and put through this snapshot.